

A MINI-PROJECT REPORT

BY

GROUP N

SUBMITTED TO

CPE 221 PRACTICAL LECTURER

BRIGHT AGBOR

GROUP MEMBER LIST

NAMES	REGISTRATION NUMBERS	GITHUB PROFILE LINKS
UDOSEN, MFONISO ANIEFIOK	23/EG/CO/064	https://github.com/mfon654
RICHARD GOD'SWILL AKPAN	23/EG/CO/069	https://github.com/godswillr
UDONTAH, EDIOMO GODWIN	23/EG/CO/050	https://github.com/01Ediomo
IBOK, SAVIOUR JOSEPH	23/EG/CO/056	https://github.com/saviour_ibok
MAC-DEDE DAVID IBINABO	23/EG/CO/065	https://github.com/cruelmac
RAYMOND ANIETIE ITORO	23/EG/CO/049	https://github.com/annyremond-del
AMBROSE AKWAIFIOK FRIDAY	23/EG/CO/051	https://github.com/akwaifiokambrose
EKONG, IME IME	23/EG/CO/062	https://github.com/imeekong
UBOM, MKPOIKANAWASI PAUL	23/EG/CO/067	https://github.com/mkpoikanawasi
BASSEY, NORA BASSEY	23/EG/CO/070	https://github.com/Ahm-nora

Introduction

This report details the development of a machine learning project aimed at predicting laptop prices. The project utilizes a dataset containing various specifications of laptops, such as brand, processor type, RAM, storage capacity, screen size, and weight. The primary objective is to build a model that can accurately estimate the price of a laptop based on its features. This type of predictive model can be valuable for consumers comparing options, retailers setting prices, or manufacturers understanding market trends.

The process followed standard data science methodologies. It began with acquiring and cleaning the raw data to ensure quality and consistency. Exploratory Data Analysis (EDA) was then performed to understand the underlying patterns, relationships, and distributions within the data. Key features were identified, and new features were engineered to potentially improve model performance. A Linear Regression model was chosen for its interpretability and simplicity. The model was trained on a portion of the data and rigorously evaluated using standard metrics like Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R-squared (R^2) on a separate test set.

During deployment on the Streamlit Community Cloud, challenges arose related to dependency management and model compatibility with the deployment environment's Python version. Specifically, initial attempts to install older package versions failed due to incompatibilities with Python 3.13. Subsequently, resolving the dependency versions allowed successful deployment, although warnings emerged regarding the scikit-learn version difference between the model training environment and the deployment environment. Furthermore, an initial mismatch between the number of features used in the deployed application and those expected by the trained model resulted in runtime errors, which were later addressed by aligning the input features in the application.

Finally, the trained model was deployed as a web application using Streamlit, making the price prediction accessible via a user-friendly interface. This report summarizes the methodology, findings, challenges encountered, and the outcomes of each stage of the project.

Data Cleaning and Preprocessing

The initial step involved loading the laptop dataset, which contained various specifications and the target price variable. A thorough inspection of the data was conducted to identify missing values, inconsistencies, and incorrect data types. Missing values were found in several columns. For numerical features such as screen size (Inches) and weight (Weight), missing entries were imputed using the median value of the respective column to maintain the central tendency and reduce the impact of potential outliers. For categorical features like company name (Company) or CPU type (Cpu), missing values were filled using the mode (most frequent value) to ensure the most common category was represented.

Outliers, which could significantly skew the model's performance, were also addressed. Statistical methods such as the Interquartile Range (IQR) were used to detect extreme values in numerical features like price (Price_euros), RAM, and weight. While outliers were identified, the decision was made to retain them in the dataset for this project to preserve the original data distribution, acknowledging that their presence might influence the model's sensitivity.

A crucial part of preprocessing was transforming categorical variables into a numerical format suitable for the machine learning model. Features with a low number of unique categories (e.g., laptop type TypeName) were encoded using One-Hot Encoding, creating binary columns for each category. This method prevents the model from assuming an ordinal relationship between categories. For features with a high cardinality, such as Company, Product, CPU, GPU, and Screen Resolution, Label Encoding was applied. This technique assigns a unique integer to each category. While this can introduce an artificial ordinal relationship, it is a common approach for handling high-cardinality features when more sophisticated methods like target encoding are not employed. The final output of this stage was a cleaned and numerically encoded dataset, ready for exploratory analysis and feature engineering.

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to gain insights into the dataset's structure, identify patterns, and understand the relationships between different features and the target variable, price. Initial visualizations included histograms and box plots for the price distribution, which revealed a right-skewed distribution, indicating that most laptops fall within a lower to mid-price range, with fewer high-end models. This skewness suggested potential benefits from log transformation for modeling purposes.

To understand the influence of categorical variables, bar charts and box plots were used to compare average prices across different categories. For instance, laptops from certain brands like Apple or high-performance gaming brands were observed to have significantly higher average prices compared to mainstream brands. Similarly, laptop types such as 'Gaming' or 'Workstation' generally commanded higher prices than 'Notebook' or 'Netbook' categories. Box plots for price by company and type confirmed these observations and highlighted the variance within each group.

Scatter plots were employed to investigate the relationship between numerical features and price. A positive correlation was observed between price and features like RAM capacity and screen size, suggesting that laptops with higher specifications tend to be more expensive. The relationship with weight was less clear, though very lightweight or very heavy laptops sometimes showed price variations. Correlation heatmaps were generated to quantify these relationships across all numerical features. This analysis highlighted the features most strongly correlated with price, which were crucial for feature selection and engineering. Key influencers identified included RAM, CPU performance indicators, GPU type (especially discrete vs. integrated graphics), storage type (SSD vs. HDD), and brand reputation. These insights guided the subsequent feature engineering process to create more predictive features.

Feature Engineering

Feature engineering was a critical step to enhance the predictive power of the model by creating new, potentially more informative features from the existing data. Several new features were derived to capture different aspects of the laptops.

First, numeric values were extracted from text-based features. For example, the RAM capacity was parsed from strings like "8GB" to a numerical value of 8. Similarly, weight and screen size were extracted from their textual representations. These extracted numerical values were then used directly or to create categorical bins.

Categorical features were created from continuous variables to group laptops into meaningful segments. Screen sizes were categorized into groups such as 'Small', 'Medium', 'Large', and 'Very Large'. Weight was grouped into categories like 'Ultra Light', 'Light', 'Medium', and 'Heavy'. RAM capacity was also binned into levels such as 'Low', 'Medium', 'High', etc. These categorical features can sometimes capture non-linear relationships better than the raw continuous values.

To reflect the market presence and potential premium associated with brand recognition, a 'Brand_Popularity' score was calculated based on the frequency count of each company in the dataset. Companies appearing more frequently were assigned a higher popularity score.

Storage type was analyzed to distinguish between HDD, SSD, and hybrid configurations. Binary flags ('Has_SSD', 'Has_HDD') were created to indicate the presence of these storage types, allowing the model to learn the price impact of different storage technologies.

Performance indicators were synthesized by combining information about the CPU and GPU brands. Simple scoring systems were used where Intel and NVIDIA processors/graphics were assigned higher scores compared to AMD or integrated solutions, respectively. A combined 'Performance_Score' was also calculated, aggregating CPU brand score, GPU brand score, and a scaled version of RAM, aiming to provide a single metric representing the overall computational and graphical capability of the laptop. These engineered features were intended to provide the linear regression model with more nuanced inputs, potentially improving its ability to predict laptop prices accurately.

Model Building

The core of the project involved building a predictive model using the processed and engineered data. The target variable was the laptop price ('Price_euros'). The dataset was split into training and testing sets using an 85-15 split (test size = 0.15), ensuring that the model's performance could be evaluated on unseen data. A random state of 24 was used for reproducibility.

The chosen model was Linear Regression. This algorithm was selected for its simplicity, interpretability, and effectiveness as a baseline for regression tasks. Linear Regression assumes a linear relationship between the input features and the target variable. The model was instantiated from the `sklearn.linear_model` library and trained on the training data (`X_train`, `y_train`) using the standard `fit` method. The training process involved finding the coefficients for each of the 17 input features and the intercept term that minimize the sum of squared residuals between the predicted and actual prices in the training set. The model was saved using `joblib` for later use in the deployment phase.

Model Evaluation

The trained Linear Regression model was evaluated on the separate test set to assess its performance on unseen data. Key metrics calculated included Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the R-squared (R^2) score. The RMSE provides a measure of the average magnitude of the errors in the predictions, with higher weight given to larger errors. The MAE represents the average absolute difference between predicted and actual prices, offering a straightforward interpretation of typical prediction error. The R^2 score indicates the proportion of variance in the laptop price that the model explains.

The model achieved an R^2 score of approximately 0.75 on the test set. This indicates that the model explains about 75% of the variance in laptop prices, which is generally considered a good performance for this type of prediction task. The RMSE and MAE values provided concrete measures of the average prediction error in Euros. While the R^2 was reasonably

high, the absolute errors (RMSE and MAE) highlighted the practical precision of the predictions.

Residual analysis was performed by plotting the residuals (actual price minus predicted price) against the predicted prices. Ideally, residuals should be randomly scattered around zero with no discernible pattern. This analysis helps check the assumptions of linear regression, such as homoscedasticity (constant variance of residuals) and linearity. The residual plot revealed some patterns, suggesting potential non-linear relationships or heteroscedasticity in the data that the simple linear model did not fully capture. Despite these limitations, the model provided a solid baseline for price prediction.

An important observation during deployment was the discrepancy between the model trained in the local environment (using scikit-learn version 1.6.1) and the model loaded in the Streamlit Cloud environment (which used scikit-learn version 1.7.1). A warning (`InconsistentVersionWarning`) was raised, indicating a potential risk of breaking code or invalid results due to the version difference. This highlights the importance of environment consistency between development and deployment stages. Although the app functioned, this version mismatch is a potential source of subtle errors or unexpected behavior.

Deployment

Deploying the trained model involved creating a user-friendly web interface using Streamlit. The first step was to ensure all necessary dependencies were correctly specified in a `requirements.txt` file for the deployment platform, Streamlit Community Cloud. Initial deployment attempts failed due to incompatible package versions, specifically older versions of `numpy` and `scikit-learn` that could not be built with the newer Python 3.13 environment used by Streamlit Cloud. This issue was resolved by updating the `requirements.txt` file to specify compatible, more recent versions of the required packages (e.g., `scikit-learn>=1.5.0`, `numpy>=1.26.0`), which allowed the cloud environment to successfully install pre-built versions (wheels) of these libraries.

A core challenge arose from a mismatch between the features collected by the Streamlit application and the features expected by the trained model. The error `ValueError: X has 8`

features, but LinearRegression is expecting 17 features as input` indicated that the application was only providing 8 user inputs, while the model required 17 features, corresponding to the features used during its training in the local environment. This discrepancy was rectified by modifying the `app.py` file to collect inputs for all 17 required features, ensuring the data fed into the model matched its training schema. The application code was structured to gather user inputs for specifications like RAM, screen size, weight, brand, CPU/GPU types, storage type, and derived performance scores, and then format them into a NumPy array with the correct order and number of elements (17) before passing it to the model's `predict` function.

Furthermore, upon successful deployment, warnings were observed regarding the model file itself. An `InconsistentVersionWarning` was raised because the model was originally trained and saved using scikit-learn version 1.6.1, while the Streamlit Cloud environment loaded it using scikit-learn version 1.7.1. Although the app functioned, this version mismatch poses a potential risk for future instability or incorrect results, highlighting the importance of consistent environments. Another `UserWarning` noted that the input data (`X`) provided by the app did not have feature names, whereas the model was fitted with feature names. These warnings were suppressed in the final `app.py` code for a cleaner user experience, but they reflect underlying technical considerations related to model persistence and data validation.

Conclusion

In conclusion, the laptop price prediction project was successfully completed, resulting in a functional web application deployed on Streamlit Community Cloud. The project followed a standard machine learning workflow, starting with data cleaning and exploratory analysis, followed by feature engineering, model building using Linear Regression, and thorough evaluation. The deployed application allows users to input laptop specifications and receive a price prediction.

However, several challenges and limitations were encountered. The most significant issues arose during the deployment phase. Dependency management proved complex due to incompatibilities between older package versions specified in `requirements.txt` and the newer Python 3.13 environment on Streamlit Cloud. This required updating the dependency

versions to ensure successful installation. A critical mismatch was also identified between the number of features the initial Streamlit app provided (8) and the number expected by the trained model (17). This was resolved by modifying the `app.py` file to collect and pass all 17 required features, ensuring the input data structure matched the model's expectations.

Furthermore, warnings were observed regarding the model file itself: an

`InconsistentVersionWarning` due to a difference between the scikit-learn versions used for training (1.6.1) and deployment (1.7.1), and a `UserWarning` about missing feature names in the input data. While these warnings were suppressed for a cleaner user interface, they highlight potential risks related to environment consistency and data validation that are important for robust model deployment.

Despite these technical hurdles, the project achieved its primary goal of creating an accessible price prediction tool. The final application demonstrates the practical application of machine learning in estimating product prices based on specifications. Future work could involve improving the model's accuracy by addressing the underlying data issues (like outliers or non-linear relationships), exploring more sophisticated algorithms, or refining the feature engineering process. Additionally, ensuring consistent environments (using tools like Docker or virtual environments with pinned versions) between development and deployment would mitigate version-related warnings and potential errors, leading to a more stable and reliable application.